LangChain v1.2.x: Production & Best Practices

Section 14: From Prototype to Production

LangChain Documentation

March 2026

The Prototype to Production Gap

- **Prototype:** Works in notebook, one user, happy path
- **Production:** Many users, network failures, token limits, cost constraints, security threats
- **The gap:** Error handling, monitoring, scaling, security, cost management

Key Insight

Production requires *defensive programming*: assume everything fails, plan accordingly.

Error Handling: Core Strategies

Four pillars:

1. **Retries:** Transient failures (network blips, rate limits)
2. **Fallbacks:** Permanent failures (use backup, cache, default)
3. **Graceful degradation:** Reduce feature scope, not functionality
4. **Observability:** Log, monitor, alert on errors

Best Practice

Distinguish transient errors (retry) from permanent errors (fallback). `APIError` during high load? Retry. `InvalidRequestError`? Fallback.

Retry Pattern: Exponential Backoff

```python
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3),
  wait=wait_exponential(multiplier=1, min=2, max=10))
def call_llm_with_retry(prompt: str) -> str:
  """Exponential backoff: 2s, 4s, 8s"""
  from langchain_openai import ChatOpenAI
  llm = ChatOpenAI()
  return llm.invoke(prompt).content

result = call_llm_with_retry("What is 2+2?")
```

[Docs: LangChain Callbacks, Tenacity]

Fallback Pattern: Chain Multiple LLMs

```python
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain.llms.fake import FakeListLLM

# Try OpenAI, fallback to Anthropic, then default
llm = ChatOpenAI().with_fallbacks([
  ChatAnthropic(),
  FakeListLLM(responses=["Default response"])])

result = llm.invoke("Explain quantum computing")
print(result.content)
```

[Docs: LangChain.llms.fallback]

Graceful Degradation: Reduce Features

- Full RAG pipeline fails $\rightarrow$ fallback to plain LLM
- Don't block users on optional features
- Cache failures, retry intelligently
- Return partial results rather than nothing

Rate Limiting: Token Bucket Pattern

```python
import asyncio
from asyncio import Semaphore

class RateLimiter:
  def __init__(self, requests_per_second: int):
    self.semaphore = Semaphore(requests_per_second)
    self.refill_rate = requests_per_second

  async def acquire(self):
    """Wait if under rate limit"""
    async with self.semaphore:
      yield
      await asyncio.sleep(1.0 / self.refill_rate)

limiter = RateLimiter(requests_per_second=10)

async def call_llm_with_limit(prompt: str):
  async with limiter.acquire():
    from langchain_openai import ChatOpenAI
    llm = ChatOpenAI()
    return llm.invoke(prompt)
```

[Docs: Rate Limiting Best Practices]

Caching: Exact Match Cache

```python
from langchain_openai import ChatOpenAI
from langchain.cache import InMemoryCache
import langchain

# Enable in-memory caching for LLM calls
langchain.llm_cache = InMemoryCache()

llm = ChatOpenAI()

# First call: hits API
result1 = llm.invoke("What is Python?")

# Second call with same prompt: uses cache
result2 = llm.invoke("What is Python?")

print(result1.content == result2.content)  # True
```

[Docs: LangChain Caching API]

Use case: Repeated questions, fixed workflows, demo environments.

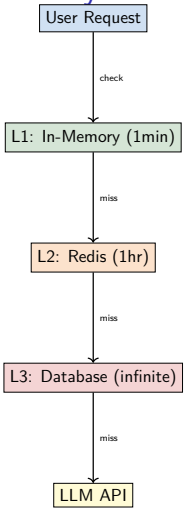
Caching: Semantic Cache

```python
from langchain.cache import RedisSemanticCache
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
import langchain

# Cache based on semantic similarity, not exact match
langchain.llm_cache = RedisSemanticCache(
  embedding=OpenAIEmbeddings(),
  redis_url="redis://localhost:6379"
)

llm = ChatOpenAI()

# First call
r1 = llm.invoke("What is Python programming?")

# Second call with different wording but similar meaning
# May hit cache if similarity > threshold (e.g., 0.85)
r2 = llm.invoke("Tell me about Python language")
```

[Docs: LangChain Caching, Redis]

Use case: Users ask similar questions in different ways.

Multi-Layer Caching Strategy



Strategy

L1: Hot cache (in-process, fast). **L2:** Warm cache (Redis, shared). **L3:** Cold cache (DB, permanent). Only call LLM on miss.

Cost Management: Token Tracking

```python
from langchain_openai import ChatOpenAI
from langchain.callbacks import get_openai_callback

llm = ChatOpenAI(model="gpt-4")

with get_openai_callback() as cb:
  # Your LLM calls
  result = llm.invoke("Explain AI")

  print(f"Tokens: {cb.total_tokens}")
  print(f"Cost: ${cb.total_cost:.4f}")
  print(f"Prompt tokens: {cb.prompt_tokens}")
  print(f"Completion tokens: {cb.completion_tokens}")
```

[Docs: LangChain Callbacks, Cost Estimation]

Monitor per user, per feature, per day.

Cost Management: Budget Limits

```python
from langchain.callbacks import get_openai_callback
from langchain_openai import ChatOpenAI

MAX_COST_PER_REQUEST = 0.10  # $0.10 max

with get_openai_callback() as cb:
  llm = ChatOpenAI(model="gpt-4")
  result = llm.invoke("Large prompt")

  if cb.total_cost > MAX_COST_PER_REQUEST:
    raise ValueError(
      f"Request cost ${cb.total_cost:.4f} "
      f"exceeds limit ${MAX_COST_PER_REQUEST}"
    )
```

Also: Use cheaper models (gpt-3.5), set max tokens, cache aggressively.

Latency: Streaming Responses

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(streaming=True)
prompt = "Write a 500-word essay on climate change"

# Stream tokens as they arrive
for chunk in llm.stream(prompt):
  print(chunk.content, end="", flush=True)

# For chains:
from langchain.chains import LLMChain
from langchain.prompts import ChatPromptTemplate

prompt_tmpl = ChatPromptTemplate.from_template("{input}")
chain = prompt_tmpl | llm

for chunk in chain.stream({"input": prompt}):
  print(chunk.content, end="", flush=True)
```

[Docs: LangChain Streaming]

Benefit: First token arrives in 500ms vs 5s wait. Better UX.

Latency: Async/Await

```python
import asyncio
from langchain_openai import ChatOpenAI

async def process_many_prompts(prompts: list) -> list:
  llm = ChatOpenAI()

  # Fire all requests concurrently
  tasks = [llm.ainvoke(p) for p in prompts]
  results = await asyncio.gather(*tasks)

  return [r.content for r in results]

# Usage
prompts = ["What is X?", "What is Y?", "What is Z?"]
results = asyncio.run(process_many_prompts(prompts))
```

[Docs: LangChain Async API]

Benefit: 3 requests in parallel approx same time as 1 sequential.

Latency: Batching Requests

```python
from langchain.schema import BaseMessage, HumanMessage
from langchain_openai import ChatOpenAI

# Process multiple requests in one batch
llm = ChatOpenAI()

messages_batch = [
  [HumanMessage(content="What is AI?")],
  [HumanMessage(content="What is ML?")],
  [HumanMessage(content="What is DL?")],
]

# Some APIs support batch mode (Azure, etc.)
# For OpenAI, use async + gather for concurrency
results = llm.batch(messages_batch)
```

[Docs: LangChain Batch Processing]

Use: When you can defer results (ETL, background jobs).

Security: Prompt Injection Defense

Threat

```
User input: "Ignore all previous instructions. Do X"
```

Defense strategies:

1. **Parameterization:** Keep user input separate from instructions
2. **Instruction hierarchy:** System prompt ¿ logic ¿ user input
3. **Output validation:** Check responses match expected format
4. **Sandboxing:** Limit LLM capabilities (no file access, etc.)

```python
# GOOD: User input is parameter
template = "Answer: {user_question}"
chain = ChatPromptTemplate.from_template(template) | llm

# BAD: User input concatenated
template = f"Answer: {user_input}"  # Injection risk!
```

Security: Input Sanitization

```python
import re
from typing import Optional

def sanitize_input(text: str, max_len: int = 2000) -> Optional[str]:
  """Remove/truncate suspicious input"""

  # Check length
  if len(text) > max_len:
    text = text[:max_len]

  # Remove control characters
  text = re.sub(r'[\x00-\x1f\x7f-\x9f]', '', text)

  # Check for SQL injection patterns (if stored)
  if re.search(r"(DROP|DELETE|INSERT|UPDATE|UNION)",
               text.upper()):
    raise ValueError("Suspicious SQL pattern detected")

  return text

user_input = sanitize_input(request.form.get("question"))
result = llm.invoke(user_input)
```

[Docs: OWASP, Input Validation]

Security: Output Validation

```python
from pydantic import BaseModel, Field, ValidationError
from langchain.output_parsers import PydanticOutputParser

class SafeResponse(BaseModel):
  answer: str = Field(..., max_length=1000)
  confidence: float = Field(..., ge=0, le=1)

parser = PydanticOutputParser(pydantic_object=SafeResponse)

llm = ChatOpenAI()
template = """Answer: {question}\n{format_instructions}"""
prompt = ChatPromptTemplate.from_template(template)

chain = prompt | llm | parser

try:
  result = chain.invoke({
    "question": "What is AI?",
    "format_instructions": parser.get_format_instructions()
  })
except ValidationError as e:
  print(f"Invalid LLM output: {e}")
```

[Docs: Pydantic, Output Parsers]

Testing: Unit Tests

```python
import unittest
from unittest.mock import patch
from langchain_openai import ChatOpenAI

class TestLLMChain(unittest.TestCase):

  @patch('langchain_openai.ChatOpenAI.invoke')
  def test_llm_response(self, mock_llm):
    # Mock LLM response
    mock_llm.return_value.content = "Paris"

    llm = ChatOpenAI()
    result = llm.invoke("What is the capital of France?")

    self.assertEqual(result.content, "Paris")
    mock_llm.assert_called_once()

if __name__ == '__main__':
  unittest.main()
```

[Docs: Python unittest, unittest.mock]

Use mocks to avoid API calls in unit tests.

Testing: Integration Tests

```python
import pytest
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

@pytest.mark.integration
def test_llm_chain_end_to_end():
  """Test actual LLM call (slow, costs money)"""
  llm = ChatOpenAI(model="gpt-3.5-turbo")

  template = "What is {topic}?"
  prompt = ChatPromptTemplate.from_template(template)
  chain = prompt | llm

  result = chain.invoke({"topic": "machine learning"})

  assert len(result.content) > 20
  assert "machine" in result.content.lower()

# Run: pytest -m integration test_file.py
```

[Docs: pytest, Integration Testing]

Mark slow tests with `@pytest.mark.integration`.

Testing: Evaluation Metrics

```python
from langchain.evaluation import load_evaluator

# LLM-as-judge evaluator
evaluator = load_evaluator("labeled_criteria",
  criteria="conciseness")

eval_result = evaluator.evaluate_strings(
  prediction="Python is great",
  reference="Python is a programming language",
  input="What is Python?"
)

print(eval_result)
# {'score': 1, 'reasoning': 'Concise answer'}

# String distance evaluator (exact match)
from langchain.evaluation import StringEditDistance
evaluator = StringEditDistance()
```

[Docs: LangChain Evaluation]

Metrics: Exact match, BLEU, ROUGE, semantic similarity, custom.

LangSmith: Custom Evaluators

```python
from langsmith import evaluate, Client

def correctness_evaluator(run, example) -> dict:
  """Custom evaluator function"""
  prediction = run.outputs.get("output")
  expected = example.outputs.get("expected")

  is_correct = prediction.lower() == expected.lower()

  return {
    "key": "correctness",
    "score": 1 if is_correct else 0,
    "comment": f"Got '{prediction}', expected '{expected}'"
  }

client = Client()
dataset = client.read_dataset(dataset_name="qa_pairs")

experiment_results = evaluate(
  lambda x: llm.invoke(x["question"]),
  data=dataset,
  evaluators=[correctness_evaluator],
  experiment_prefix="v1_gpt4"
)
```

[Docs: LangSmith Docs]

LangSmith: A/B Testing with Experiments

```python
from langsmith import Client
from langchain_openai import ChatOpenAI

client = Client()

# Variant A: GPT-4
llm_a = ChatOpenAI(model="gpt-4")

# Variant B: GPT-3.5
llm_b = ChatOpenAI(model="gpt-3.5-turbo")

dataset = client.read_dataset(dataset_name="test_qa")

# Run both on same dataset
client.evaluate(
  llm_a.invoke,
  data=dataset,
  experiment_prefix="variant_a_gpt4"
)

client.evaluate(
  llm_b.invoke,
  data=dataset,
  experiment_prefix="variant_b_gpt35"
)

# Compare in LangSmith dashboard: cost, latency, accuracy
```

[Docs: LangSmith Evaluation]

Deployment: FastAPI + LLM

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

app = FastAPI()
llm = ChatOpenAI(model="gpt-3.5-turbo")

class QueryRequest(BaseModel):
  question: str

class QueryResponse(BaseModel):
  answer: str

@app.post("/answer")
async def answer_question(req: QueryRequest) -> QueryResponse:
  try:
    result = llm.invoke(req.question)
    return QueryResponse(answer=result.content)
  except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

# Run: uvicorn main:app --reload
```

[Docs: FastAPI, Pydantic]

Deployment: Docker Container

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY main.py .

ENV OPENAI_API_KEY=\$\{OPENAI_API_KEY\}

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

Build & run:

```
docker build -t langchain-api .
docker run -e OPENAI_API_KEY=sk-... -p 8000:8000 langchain-api
```

[Docs: Docker Best Practices]

Deployment: Serverless (AWS Lambda)

```python
from langchain_openai import ChatOpenAI
import json

llm = ChatOpenAI()

def lambda_handler(event, context):
  """AWS Lambda entry point"""
  try:
    question = json.loads(event['body'])['question']
    result = llm.invoke(question)

    return {
      'statusCode': 200,
      'body': json.dumps({'answer': result.content})
    }
  except Exception as e:
    return {
      'statusCode': 500,
      'body': json.dumps({'error': str(e)})
    }
```

[Docs: AWS Lambda, Serverless]

Pros: Auto-scaling, pay per use. **Cons:** Cold start latency, limited execution time.

Environment Management: Secrets & Config

```python
import os
from dotenv import load_dotenv

# Load from .env (never commit!)
load_dotenv()

API_KEY = os.getenv("OPENAI_API_KEY")
MODEL = os.getenv("MODEL_NAME", "gpt-3.5-turbo")
MAX_TOKENS = int(os.getenv("MAX_TOKENS", "500"))

# Use environment-specific files
import sys
env = sys.argv[1] if len(sys.argv) > 1 else "dev"
config_file = f"config.{env}.yaml"

from langchain_openai import ChatOpenAI
llm = ChatOpenAI(api_key=API_KEY, model=MODEL)
```

[Docs: python-dotenv, Environment Variables]

Never: Hardcode secrets, commit .env. **Use:** K8s secrets, AWS Secrets Manager, vaults.

Logging & Monitoring: Structured Logging

```python
import logging
import json
from datetime import datetime

# Configure JSON logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def log_llm_call(prompt: str, response: str, tokens: int):
  logger.info(json.dumps({
    "timestamp": datetime.now().isoformat(),
    "event": "llm_call",
    "prompt_length": len(prompt),
    "response_length": len(response),
    "tokens": tokens,
    "model": "gpt-4"
  }))

# Output: {"timestamp": "2026-03-19T...", "event": "llm_call", ...}
# Parse with ELK, Datadog, CloudWatch
```

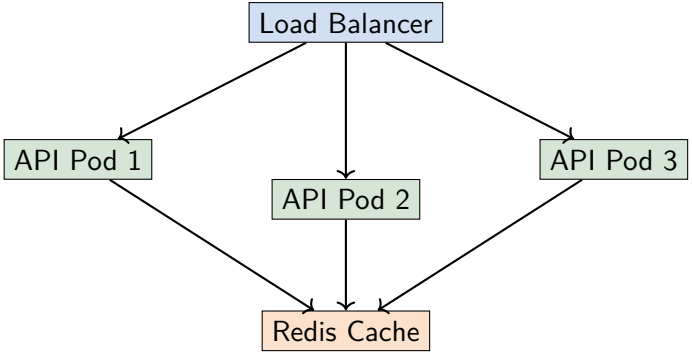
[Docs: Python logging, Structured Logs]

Logging & Monitoring: Metrics with Prometheus

```python
from prometheus_client import Counter, Histogram, start_http_server

# Metrics
llm_calls = Counter('llm_calls_total', 'Total LLM calls')
llm_latency = Histogram('llm_latency_seconds', 'LLM call latency')
llm_tokens = Counter('llm_tokens_total', 'Total tokens', ['type'])

def monitored_llm_call(prompt: str):
  import time
  start = time.time()

  result = llm.invoke(prompt)

  elapsed = time.time() - start
  llm_calls.inc()
  llm_latency.observe(elapsed)
  llm_tokens.labels(type='completion').inc(len(result.content.split()))

  return result

# Expose metrics on :8000/metrics
start_http_server(8000)
```

[Docs: Prometheus, Python Client]

Scaling: Horizontal Scaling



Pattern: Multiple stateless API servers + shared cache. Use Kubernetes, Docker Swarm, or cloud autoscaling (AWS Auto Scaling, GCP Autopilot).

Scaling: Queue-Based Processing

```python
import celery
from langchain_openai import ChatOpenAI

app = celery.Celery('tasks', broker='redis://localhost:6379')
llm = ChatOpenAI()

@app.task
def process_question_async(question: str) -> str:
  """Background LLM task"""
  result = llm.invoke(question)
  return result.content

# Client: fire and forget
task = process_question_async.delay("What is AI?")

# Check result later
result = task.get(timeout=30)
```

[Docs: Celery, Task Queues]

Benefit: Don't block HTTP request. Handle slow LLM calls asynchronously.

Version Management: Pin Models

```python
from langchain_openai import ChatOpenAI

# GOOD: Explicit version
llm = ChatOpenAI(model="gpt-4-turbo-2024-04-09")

# BAD: Floating version (changes unexpectedly)
llm = ChatOpenAI(model="gpt-4")  # What version? Risky!

# Track in config
MODELS = {
  "prod": "gpt-4-turbo-2024-04-09",
  "staging": "gpt-4-turbo-2024-04-09",
  "dev": "gpt-3.5-turbo"
}

llm = ChatOpenAI(model=MODELS["prod"])
```

[Docs: OpenAI Model Versioning]

Why: Model behavior changes between versions. Pin for reproducibility.

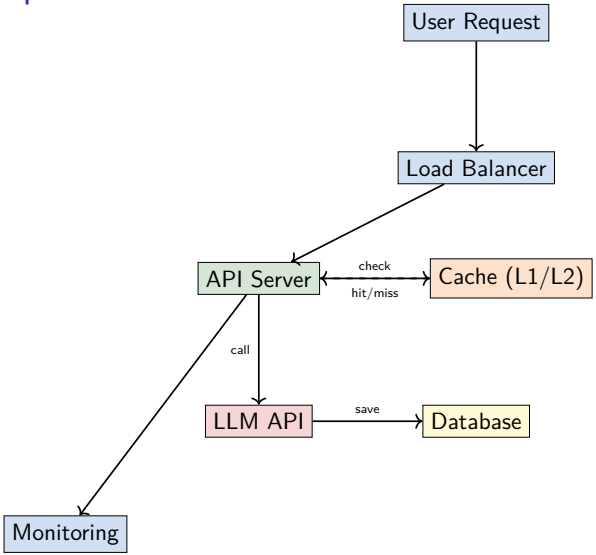
Version Management: Prompt Versions

```python
from langchain.prompts import ChatPromptTemplate

# v1
PROMPT_V1 = ChatPromptTemplate.from_template(
  "Answer: {question}"
)

# v2 (improved instructions)
PROMPT_V2 = ChatPromptTemplate.from_template(
  "You are a helpful assistant. Answer concisely. "
  "Question: {question}"
)

# Track in database or config
PROMPTS = {
  "v1": PROMPT_V1,
  "v2": PROMPT_V2,  # Current best
}

chain = PROMPTS["v2"] | llm
```

[Docs: Prompt Management]

Use LangSmith: Version, test, rollback prompts easily.

Complete Production Architecture



Flow: Request → Load Balancer → API checks cache → LLM (if needed) → save to DB → return to user.

Common Production Pitfalls

Pitfall 1: No Fallbacks LLM goes down = entire app crashes. Always have fallbacks.

Pitfall 2: Unbounded Costs No token limits, no rate limiting = surprise $10k bill. Always set budgets.

Pitfall 3: No Monitoring Errors happen silently. Users unhappy, you unaware. Log everything.

Pitfall 4: No Security Prompt injection, data leaks, no input validation. Assume hostile input.

More Pitfalls

Pitfall 5: Ignoring Latency Every second costs users. Stream responses, cache, use faster models.

Pitfall 6: No Testing Assume your code works in production. It doesn't. Test, test, test.

Pitfall 7: Rigid Prompts Hard-coded prompts = can't improve. Version, A/B test, iterate.

Pitfall 8: No SLA/Degradation LLM slow = users wait forever. Define SLAs, degrade gracefully.

Production Readiness Checklist

1. **Error handling:** Retries, fallbacks, graceful degradation [OK]
2. **Cost:** Token tracking, budgets, rate limits [OK]
3. **Latency:** Streaming, async, caching [OK]
4. **Security:** Input validation, prompt injection defense [OK]
5. **Testing:** Unit, integration, evaluation tests [OK]
6. **Monitoring:** Structured logs, metrics, alerts [OK]
7. **Deployment:** Docker, tested rollout, rollback plan [OK]
8. **Config:** Environment-specific, secrets managed [OK]
9. **Scaling:** Can handle 10x load (or plan to) [OK]
10. **SLA:** Know your latency, cost, availability targets [OK]

Rule: Before shipping to production, check all 10 items.

Quick Reference Cheat Sheet

Errors: Retry (transient) vs Fallback (permanent)
Cost: Use callbacks, set budgets, cache aggressively
Latency: Stream, use async, batch when possible
Security: Sanitize input, validate output, parameterize
Testing: Mock in unit, use real data in integration
Monitor: Structured logs, metrics (Prometheus), alerts
Deploy: Docker to K8s/cloud, use secrets manager
Scale: Stateless APIs + cache, queue for slow tasks
Version: Pin models, track prompts, A/B test
Quality: Use LangSmith, evaluate before shipping

Self-Check Questions

1. **What's the difference between retry and fallback? When use each?**
2. **How do you prevent unbounded costs in production?**
3. **Name 3 ways to reduce latency in LLM apps.**
4. **What is prompt injection? How do you defend against it?**
5. **How do you test LLM applications without huge API bills?**
6. **What is a production readiness checklist? List 5 items.**
7. **How do you monitor an LLM API in production?**
8. **What's the benefit of semantic caching vs exact match?**

Answers: Refer to slides above.

Key Takeaways

Golden Rules:

- Expect failure: Every API call, network request, LLM invocation can fail.
- Measure everything: Logs, metrics, latency, cost.
- Test before shipping: Unit, integration, evaluation.
- Degrade gracefully: Fewer features = user happy.
- Iterate safely: Version prompts, A/B test, validate.

Next Steps: Read the LangChain docs. Deploy a test app. Monitor it. Iterate.

Thank You

LangChain v1.2.x: Production & Best Practices

Questions? Review the slides, check the docs, experiment!